# 30 TIÊU CHÍ MSA Postman tối giản

## URL. Body. Token. Không Collection, không Environment, không script trong Postman.

**Kết quả chạy lại ngày 27/10/2026:** 31/31 full E2E checks, phủ 30/30 tiêu chí. Luồng test tự động gọi Gateway và kiểm tra DB/RabbitMQ bằng dữ liệu riêng; đây **không phải** tuyên bố đã bấm tay toàn bộ 30 mục trong Postman.

Tài liệu này chuyển các API hiện tại thành thao tác bấm tay để demo. Các mục source/Docker/RabbitMQ/DB dùng đúng công cụ cần thiết; Postman một mình không thể chứng minh mã hóa ở DB hay kiến trúc source.

| Công cụ | Chuẩn bị tối thiểu |
|---|---|
| Postman | New HTTP Request; để No Environment. Body có JSON: chọn raw rồi JSON. Body không cần: chọn none. |
| Docker Desktop | Stack đã chạy; đợi 13 containers healthy. Không recreate/reset khi đang PASS. |
| Git Bash / VS Code | Chỉ xem cấu hình, source, log và query minh chứng. Không mở secret trước màn hình chấm. |
| RabbitMQ | http://localhost:15672; dùng credential local của bạn, không ghi mật khẩu vào PDF. |

## Base URL duy nhất: http://localhost:8080/api

**Quy ước nhập:** dùng field ngắn trong body; response có nhãn đầy đủ. Ví dụ nhập **vt**, nhưng đọc **Vehicle Type**; nhập **s**, đọc **Status**. Không thêm /v1.

**Quy ước ID:** {cid}, {did}, {bid}, {oid}, {tid}, {pid} là chỗ thay bằng số thật lấy từ response. Không gõ dấu ngoặc. Không tạo biến Environment. Ví dụ bid=60 thì gõ /trips/by-booking/60.

**Thông số demo:** OTP=123; amount=50000; radius=1 km; BIKE mặc định; reason=x; star=5; comment=ok; rate limit=3 request/10 giây.

**Giới hạn an toàn:** dữ liệu giả; khóa AES mẫu công khai chỉ dành cho demo. Không xóa volume, database, queue hoặc dữ liệu cũ. Nếu broker bị restart riêng, cần kiểm tra consumer count trước khi demo.

THAO TÁC DEMO

# Chuẩn bị 3 token, rồi đi đúng thứ tự

| Ký hiệu trong PDF | Login / cách dùng |
|---|---|
| C - Customer | c@c.com / 123; dùng cho Booking, Trip của mình, Payment, Review. |
| A - Admin | a@a.com / 123; dùng xem Driver, Nearby và duyệt hồ sơ. |
| D - Driver BIKE | bike@cab.demo / 123; đã APPROVED/AVAILABLE gần điểm demo. Driver CAR d@d.com không nhận Booking BIKE. |
| DN / CN | Token của Driver/Customer vừa đăng ký, chỉ dùng các bước riêng có ghi DN/CN. |

Có thể giữ 4 tab chưa lưu: No Auth, C, A, D. Mỗi tab đặt Authorization một lần rồi đổi method/URL/body khi cần. Không cần đặt tên, lưu request hoặc nhập token lại cho mỗi request cùng role.

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/login
{"email":"c@c.com","pw":"123"}
```

**Đạt:** HTTP 200; copy giá trị Token. Ghi Customer ID từ login.

Đổi email để lấy A và D. Authorization > Bearer Token > dán JWT thật. **Không gõ chữ C/A/D vào ô Token.** Không copy dấu nháy quanh JWT.

| Thứ tự tối ưu | Mục chấm / việc làm |
|---|---|
| 1. Kiểm tra nền | 01-08: source, Git, Compose, health, RabbitMQ và ingress. |
| 2. Dữ liệu đầu vào | 09-14: đăng ký thử, login, profile/Driver/Nearby/history. Giữ C/A/D seed cho luồng chính. |
| 3. Chuyến T1 | 15 → 16 → 17 → 19 + 30 → 20. Hoàn thành rồi thanh toán/replay, đánh giá star5/c=ok. |
| 4. Chuyến T2 | Tạo Booking/Accept mới, rồi làm 18 để hủy ở ASSIGNED. Không hủy chuyến T1. |
| 5. Hồ sơ mới | 21 → 22 → 23 với DN. Kết thúc DN OFFLINE để không tranh Offer của D. |
| 6. Bảo mật | 24, 25, 27, 28, 29; 26 dùng chuyến T3 đã hoàn thành và chưa Review. |

**Chống 429 ngoài ý muốn:** mọi role dùng chung giới hạn theo IP. Nếu bị 429 ở mục khác, chờ 10-11 giây rồi gửi lại. Chỉ mục 29 cố ý bấm liên tiếp. ID mới phải ghi lại trên giấy hoặc copy từ response.

# 01-05 / Source, Git và Compose

## 01. Mô tả kiến trúc source code

Bước 1: mở VS Code Explorer > services. Bước 2: chỉ 8 thư mục business và api-gateway; mở infrastructure, proto, contracts/events.

```
ls services
ls infrastructure
ls proto
ls contracts/events
```

**Đạt:** Auth, Customer, Driver, Booking, Trip, Payment, Notification, Review; 1 Gateway; 4 thành phần hạ tầng.

## 02. .gitignore và .env trên Git

```
git check-ignore .env
git ls-files .env
git status --short
```

**Đạt:** lệnh đầu in .env; lệnh thứ hai không in gì. Mở .gitignore cho thấy .env, backups, node_modules bị ignore; chỉ có .env.example là cấu hình mẫu. Không mở .env thật.

## 03. Nhiệm vụ Gateway

Mở services/api-gateway/src/routes và middleware. Nói ngắn: “Gateway nhận REST, kiểm tra JWT/RBAC, rate limit bằng Redis và gọi business services qua gRPC.” Login ở mục 10; so role ở mục 12/28; rate ở mục 29 là minh chứng chạy.

## 04. IPC: gRPC và RabbitMQ

```
Booking -> Customer / Driver
Trip    -> Driver
Review  -> Trip
```

Chỉ đúng 4 cạnh domain. Tại mục 15-16, Booking gọi Customer/Driver đồng bộ; driver.accepted qua RabbitMQ tạo Trip. Mở proto/ và contracts/events/ để chỉ contract; không tạo Booking dư chỉ để trình bày mục này.

## 05. Compose và đủ 13 containers

```
docker compose config --quiet
docker compose config --services
docker compose ps
```

**Đạt:** 13 thành phần running/healthy. Stack đang PASS thì chỉ kiểm tra; không chạy up/recreate, down -v hoặc reset.

# 06-08 / Health, RabbitMQ, Gateway-only

## 06. Health / Readiness / 8 services

**GET** | Authorization: **No Auth** | Body: none

`http://localhost:8080/api/health`

**Đạt:** 200; status=HEALTHY.

**GET** | Authorization: **No Auth** | Body: none

`http://localhost:8080/api/ready`

**Đạt:** 200; status=HEALTHY khi đủ dependency.

**GET** | Authorization: **No Auth** | Body: none

`http://localhost:8080/api/health/services`

**Đạt:** 200; đúng 8 service đều HEALTHY, không Report/Audit.

## 07. RabbitMQ đang publish/consume

Bước 1: mở http://localhost:15672 và đăng nhập bằng tài khoản RabbitMQ local. Bước 2: Exchanges > cab.events, kiểm tra type=topic và durable. Bước 3: Queues, kiểm tra 4 queue dưới đây có consumer. Bước 4: sau mục 15/16/17/19, đối chiếu side effect trên Payment/Trip/Notification; Messages có thể về 0 vì đã consume, không phải mất event.

| Queue | Nhận event |
|---|---|
| cab.payment.q | booking.created; trip.completed |
| cab.trip.q | driver.accepted; payment.completed |
| cab.driver.q | trip.canceled; trip.completed |
| cab.notification.q | offer.created; driver.approval.changed; trip.status.changed; trip.canceled; payment.completed |

## 08. Business request chỉ đi qua Gateway

Mở docker-compose.yml phần ports: Gateway 8080; RabbitMQ Management 15672. Business services không publish host port. Tất cả URL nghiệp vụ trong PDF đi localhost:8080/api. Dùng expose nội bộ không đồng nghĩa public host port.

**Câu nói:** “Client chỉ biết Gateway; gRPC và database nằm trong Docker network.”

# 09-12 / Customer, login và Driver detail

## 09. Đăng ký Customer mới

Dùng email chưa tồn tại; nếu trùng, tăng số c27 → c28. Không xóa account cũ để chạy lại.

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/register
{"email":"c27@c.com","pw":"123","name":"An"}
```

**Đạt:** 201; User ID và Customer ID mới. Có thể thêm phone, không bắt buộc và không cần OTP.

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/login
{"email":"c27@c.com","pw":"123"}
```

**Đạt:** 200; account mới đăng nhập được. Token này là CN, chỉ để chứng minh đăng ký.

## 10. Đăng nhập Customer dùng cho luồng chính

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/login
{"email":"c@c.com","pw":"123"}
```

**Đạt:** 200; Token, User ID, Role=CUSTOMER, Customer ID. Giữ token này làm C để dùng dữ liệu history có sẵn.

## 11. Xem thông tin Customer bằng mã số

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/customers/{cid}
```

**Đạt:** 200; Customer ID đúng từ login C. Nếu profile có phone thì chủ sở hữu thấy Phone; không có password/hash.

## 12. Xem Driver bằng mã số với Admin

Login a@a.com / 123 để lấy A. Login bike@cab.demo / 123 để lấy D và Driver ID; dùng did đó cho mục 12 và 28, không đoán từ User ID.

**GET** | Authorization: **Admin** | Body: none

```
http://localhost:8080/api/drivers/{did}
```

**Đạt:** 200; Driver ID, Name, Approval Status, Availability Status, Vehicle Type, License Plate.

> User ID ≠ Customer ID ≠ Driver ID. Luôn lấy đúng nhãn trong response. Ví dụ cid=3 thì URL /customers/3, không lấy uid=13 thay vào.

THAO TÁC DEMO

# 13-15 / Nearby, history và Booking

## 13. Nearby: 1 km, limit và paging

**GET** | Authorization: **Admin** | Body: none

```
http://localhost:8080/api/drivers/nearby?lat=10.76&lng=106.68
```

**Đạt:** 200; mặc định r=1, page=1, limit=5. Dữ liệu demo có ít nhất 5 Driver với nhiều trạng thái.

**GET** | Authorization: **Admin** | Body: none

```
http://localhost:8080/api/drivers/nearby?lat=10.76&lng=106.68&page=2&limit=2
```

**Đạt:** Đổi trang/limit có tác dụng. Có thể thêm &r=1; Distance (km) ≤1.

Nearby được nhìn nhiều trạng thái; Matching chỉ APPROVED + AVAILABLE + đúng loại xe. Lat/lng luôn bắt buộc, không có tọa độ mặc định.

## 14. Booking của Customer: limit và paging

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/customers/me/bookings?page=1&limit=2
```

**Đạt:** 200; 2 Booking đầu của đúng Customer. Seed C có ≥5 Booking. Đổi page=2 để chỉ phân trang.

## 15. Đặt xe mặc định BIKE

Trước khi đặt: D phải APPROVED, AVAILABLE, BIKE và gần 10.76/106.68. D seed được chuẩn bị sẵn; nếu đang BUSY, hoàn tất/hủy đúng chuyến hoặc kiểm tra, không ép DB về AVAILABLE.

**POST** | Authorization: **Customer** | Body: raw / JSON

```
http://localhost:8080/api/bookings
{"p":{"lat":10.76,"lng":106.68},"d":{"lat":10.77,"lng":106.69}}
```

**Đạt:** 201; Vehicle Type=BIKE, Status=SEARCHING, có Booking ID/Offer ID/Driver ID. Ghi bid, oid và did được chọn.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/payments/by-booking/{bid}
```

**Đạt:** Cùng Booking có một Payment: Amount=50000, Status=PENDING, Eligible=false, Trip ID=null. Ghi pid.

Muốn CAR: thêm "vt":"CAR" và dùng Driver CAR đủ điều kiện. Nếu NO_DRIVER_FOUND, đừng bấm tạo liên tục: kiểm tra loại xe/approval/online/location. Một Booking chỉ tạo một Offer.

# 16 / Driver nhận chuyến, Customer thấy Driver

## 16. Thông báo → Offer → Accept → Trip

1. Dùng **đúng Driver ID trong Offer**. D seed phải là BIKE đã chọn; không dùng token CAR d@d.com cho Offer BIKE. Nếu Driver ID khác, lấy đúng tài khoản tương ứng trước khi Accept.

**GET** | Authorization: **Driver BIKE nhận Offer** | Body: none

```
http://localhost:8080/api/notifications?limit=20
```

**Đạt:** Có OFFER_CREATED, Reference=OFFER:{oid}; notification thuộc Driver đang login.

**GET** | Authorization: **Driver BIKE nhận Offer** | Body: none

```
http://localhost:8080/api/drivers/me/offers
```

**Đạt:** Tìm đúng oid/bid của mục 15, trạng thái OPEN.

**POST** | Authorization: **Driver BIKE nhận Offer** | Body: none

```
http://localhost:8080/api/offers/{oid}/accept
```

**Đạt:** 200; Status=ACCEPTED. Một Assignment, Driver chuyển BUSY. Body none, không gửi did tùy ý.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/trips/by-booking/{bid}
```

**Đạt:** 200; Trip ASSIGNED, Paid=false; có trường Driver với Driver ID/Name/Vehicle Type/License Plate. Ghi tid.

Nếu Trip chưa xuất hiện ngay, chờ ngắn rồi gửi lại GET vì RabbitMQ xử lý bất đồng bộ. Không Accept Offer khác hoặc tạo Booking mới để thay thế.

```
Ví dụ phần bổ sung trong response /api:
"Driver": {
  "Driver ID": 52,
  "Name": "Driver BIKE Demo",
  "Vehicle Type": "BIKE",
  "License Plate": "59B00099"
}
```

Các số/tên trên chỉ minh họa. Customer không cần quyền gọi Driver Admin endpoint để thấy Driver của Trip mình. Dùng Customer khác đọc Trip này phải 403; không mở quyền admin để demo.

# 17 / Chuyến đi đúng state và có location

## 17. T1: ASSIGNED → ARRIVED → IN_PROGRESS → COMPLETED

Dùng token D đã Accept đúng chuyến, giữ nguyên tid. Không bỏ qua state. Location phải cập nhật **sau IN_PROGRESS, trước COMPLETED**.

**PATCH** | Authorization: **Driver BIKE nhận Offer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid}/status
{"s":"ARRIVED"}
```

**Đạt:** 200; Status=ARRIVED.

**PATCH** | Authorization: **Driver BIKE nhận Offer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid}/status
{"s":"IN_PROGRESS"}
```

**Đạt:** 200; Status=IN_PROGRESS.

**PATCH** | Authorization: **Driver BIKE nhận Offer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid}/location
{"lat":10.765,"lng":106.685}
```

**Đạt:** 200; lưu tọa độ, không đổi Status/Paid.

**PATCH** | Authorization: **Driver BIKE nhận Offer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid}/status
{"s":"COMPLETED"}
```

**Đạt:** 200; Status=COMPLETED, Paid=false. trip.completed được phát sau persistence.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/payments/by-booking/{bid}
```

**Đạt:** Vẫn pid cũ, nay có Trip ID, Eligible=true, Status vẫn PENDING.

**Đối chứng:** từ ASSIGNED gửi thẳng COMPLETED phải bị từ chối. Driver khác không được đổi Trip; COMPLETED/CANCELED là terminal. Không cần thực hiện các nhánh lỗi này khi chỉ còn ít thời gian.

Trip hoàn thành không tự tạo Payment mới và không tự paid=true. Tiếp tục ngay trang 9 để thanh toán T1; không hủy T1 ở mục 18.

# 19 + 30 / Payment và replay cùng response

## 19. Pay hợp lệ trước, callback sau

Dùng C sở hữu Payment, pid lấy từ Payment by Booking. Trip phải COMPLETED và Payment Eligible=true. Một lần Pay chỉ ghi nhận yêu cầu; callback mock mới hoàn tất.

**POST** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/payments/{pid}/pay
Header: Idempotency-Key: PM{pid}
```

**Đạt:** 200; PENDING, cùng Payment ID, có Idempotency Key. Giữ response này làm bản gốc.

Ví dụ pid mới=60 thì gõ key PM60. Không gõ dấu ngoặc. P1 đã dùng cho Payment cũ trên máy này; không lấy P1 cho Payment mới. Có thể gửi body {"amt":50000}, nhưng bỏ body ngắn hơn.

## 30. Replay trước và sau callback

Bấm Send lại **y hệt** request Pay: cùng URL, token, key và body. Không đổi pid hay amount.

**Đạt:** cùng response Pay ban đầu, cùng pid; không thêm Payment hay xử lý lần hai.

**POST** | Authorization: **Customer** | Body: raw / JSON

```
http://localhost:8080/api/payments/{pid}/callback
{"s":"COMPLETED"}
```

**Đạt:** 200; Payment COMPLETED; phát payment.completed một lần.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/payments/{pid}
```

**Đạt:** Status=COMPLETED.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/trips/{tid}
```

**Đạt:** Status=COMPLETED, Paid=true.

Quay lại request Pay cũ, Send lần nữa: vẫn trả snapshot **PENDING** đúng như Pay lần đầu. Đó là replay đúng thiết kế, không phải Payment bị quay ngược trạng thái; GET ở trên mới là trạng thái hiện tại.

Callback trước Pay hợp lệ →409. Cùng key nhưng body đổi thành {"amt":1} →409. Gửi lại callback không được phát thêm payment.completed. Không dùng Redis để idempotency Payment.

# 20 và 18 / Review T1, hủy chuyến T2 riêng

## 20. Review chuyến đã hoàn thành

**POST** | Authorization: **Customer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid}/reviews
{"star":5,"c":"ok"}
```

**Đạt:** 201; Review ID, đúng Trip ID/Driver ID, Rating=5, Comment=ok.

Gửi lại nguyên request: **409**, không tạo Review trùng. Sai Customer không được review; star ngoài 1-5 bị từ chối. Giữ T1 đã review cho minh chứng, không dùng nó ở test XSS.

## 18. Tạo T2 mới để hủy, không dùng T1 COMPLETED

1. Khi D đã AVAILABLE sau T1, lặp **POST /bookings** với p/d ở trang 6. Ghi bid2/oid2. 2. D Accept oid2. 3. C GET Trip by Booking bid2, ghi tid2, kiểm tra ASSIGNED. Đây là nhánh hủy riêng.

**POST** | Authorization: **Customer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid2}/cancel
{"r":"x"}
```

**Đạt:** 200; Status=CANCELED, Cancel Reason=x. Không cần đổi state trước khi hủy ASSIGNED.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/notifications?limit=20
```

**Đạt:** Có TRIP_CANCELED, Reference=TRIP:{tid2}.

**GET** | Authorization: **Driver BIKE nhận Offer** | Body: none

```
http://localhost:8080/api/notifications?limit=20
```

**Đạt:** Driver cũng có TRIP_CANCELED cho cùng tid2. Hai người nhận, không phải một document dùng chung uid.

Driver trở lại AVAILABLE. Gửi lại cancel hoặc hủy T1 COMPLETED phải bị từ chối. Notification lặp event không nhân đôi (unique theo eid+uid).

Một Booking/Payment của chuyến hủy có thể còn trong DB; không xóa để “dọn demo”. Payment không eligible thì không thanh toán. Đây là dữ liệu kiểm thử riêng, không phải yêu cầu tạo Refund workflow.

# 21-22 / Phone, OTP và Admin duyệt hồ sơ

## 21. Đăng ký Driver mới - dùng phone chưa tồn tại

Không cần nhập vt: mặc định BIKE. Dùng bộ dữ liệu mới mỗi lượt: d27 → d28; phone/plate đổi tương ứng. OTP phải cùng phone, trong 300 giây.

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/drivers/otp
{"phone":"0901027001"}
```

**Đạt:** 200; OTP=123 (mock, không gửi SMS).

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/drivers/register
{"phone":"0901027001","email":"d27@d.com","pw":"123",
 "otp":"123","name":"Driver Demo","plate":"59B27001"}
```

**Đạt:** 201; Driver ID mới, User ID mới, PENDING_APPROVAL. Ghi did_new; chưa được nhận chuyến.

OTP sai/hết hạn/phone chưa request phải bị từ chối trước khi tạo Auth Account. Gửi lại cùng hồ sơ khi mạng gián đoạn là resume cùng UID/DID, không tạo mới. Customer phone thì tùy chọn, không OTP.

## 22. Admin list → detail → approve → Driver nhận kết quả

**GET** | Authorization: **Admin** | Body: none

```
http://localhost:8080/api/drivers/pending-approval?page=1&limit=20
```

**Đạt:** Tìm did_new trong danh sách pending.

**GET** | Authorization: **Admin** | Body: none

```
http://localhost:8080/api/drivers/{did_new}
```

**Đạt:** Đúng hồ sơ vừa đăng ký; không nhầm User ID.

**PATCH** | Authorization: **Admin** | Body: raw / JSON

```
http://localhost:8080/api/drivers/{did_new}/approval
{"ap":"APPROVED"}
```

**Đạt:** 200; APPROVED. Muốn test REJECTED phải dùng hồ sơ pending khác, không đổi qua lại hồ sơ đã duyệt.

Login d27@d.com / 123 lấy DN, rồi GET /api/notifications?limit=20 bằng DN: có DRIVER_APPROVAL_CHANGED cho chính Driver mới. Chờ ngắn nếu consumer chưa cập nhật. Không dùng D seed để xem notification DN.

# 23-24 / Online/Offline và mã hóa lưu trữ

## 23. Driver mới cập nhật location, Online rồi Offline

**PUT** | Authorization: **Driver mới** | Body: raw / JSON

```
http://localhost:8080/api/drivers/me/location
{"lat":10.76,"lng":106.68}
```

**Đạt:** 200; giữ approval/profile, cập nhật tọa độ hiện tại.

**PATCH** | Authorization: **Driver mới** | Body: raw / JSON

```
http://localhost:8080/api/drivers/me/availability
{"on":true}
```

**Đạt:** 200; Availability Status=AVAILABLE, Approval Status=APPROVED.

**PATCH** | Authorization: **Driver mới** | Body: raw / JSON

```
http://localhost:8080/api/drivers/me/availability
{"on":false}
```

**Đạt:** 200; OFFLINE. Kết thúc DN OFFLINE để không tranh Offer của D seed.

Pending/Rejected bật Online phải bị từ chối; BUSY không được tùy ý Offline. Nếu muốn dùng DN nhận chuyến mới, bật lại Online và đảm bảo đó là Driver gần nhất; luôn nhìn Driver ID trong Offer.

## 24. Encryption at rest - cần query DB, không chỉ Postman

Trong Git Bash chạy từng lệnh dưới; -T tránh lỗi TTY, chỉ đọc, không UPDATE/DROP. cab là user PostgreSQL canonical. Password là bcrypt hash, không phải plaintext hoặc mã hóa giải ngược.

```
docker compose exec -T postgres psql -U cab -d auth_db \
  -c "SELECT uid,left(pw_hash,20) AS hash_prefix
      FROM user_account ORDER BY uid LIMIT 3;"
```

```
docker compose exec -T postgres psql -U cab -d driver_db \
  -c "SELECT did,left(lic_enc,45) AS ciphertext,key_ver
      FROM driver_profile WHERE lic_enc IS NOT NULL LIMIT 3;"
```

**Đạt:** bcrypt prefix, ciphertext v1:nonce:ciphertext:tag, không có license gốc. Hồ sơ mới không nhập giấy phép có thể NULL; chọn hàng có lic_enc để minh chứng. Key đọc qua DATA_ENCRYPTION_KEY_BASE64 trong ENV; không mở/in key.

Khóa mẫu hiện dùng cho local demo đã công khai theo thiết kế seed. Chỉ dùng dữ liệu giả; không gọi đó là key production bí mật và không thay key trực tiếp vì dữ liệu cũ sẽ không giải mã được.

THAO TÁC DEMO

# 25, 27-29 / Bảo mật nhanh trong Postman

## 25. SQL Injection không bypass Login

**POST** | Authorization: **No Auth** | Body: raw / JSON

```
http://localhost:8080/api/login
{"email":"' OR 1=1 --","pw":"x"}
```

**Đạt:** HTTP 400 hoặc 401; không có Token, không lộ SQL/DB. 400 từ validation cũng là chặn hợp lệ.

## 27. JWT tampering trả 401

Copy C sang ô Bearer của một tab test. Đổi **một ký tự ở giữa phần chữ ký sau dấu chấm thứ hai**, giữ nguyên hai phần đầu. Không chỉ đổi ký tự cuối vì có thể rơi vào bit base64 không có ý nghĩa.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/customers/{cid}
```

**Đạt:** Với token đã sửa: 401. Xong khôi phục token C thật; không lưu token giả đè lên ghi chú của mình.

## 28. Unauthorized API: cùng Driver URL khác role

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/drivers/{did}
```

**Đạt:** 403 Forbidden, không trả Driver data. Cùng URL với A ở mục 12 trả 200.

## 29. Rate limit Redis: 3 request / 10 giây

Chờ ít nhất 11 giây không gọi business API. Dùng C thật; cùng tab và cùng URL dưới đây, bấm Send 4 lần nhanh trong 10 giây. Không chạy stress test 1000 request/giây.

**GET** | Authorization: **Customer** | Body: none

```
http://localhost:8080/api/vehicle-types
```

**Đạt:** Ba request đầu 200; request thứ tư 429 Too Many Requests. Nếu rate còn từ trước, chờ rồi làm lại.

Gọi /api/health vẫn 200; health được miễn rate limit. Chờ hết cửa sổ trước khi sang mục khác. Chỉ 429 ở bài rate limit mới là kết quả mong đợi.

# 26 / XSS riêng và xử lý lỗi khi demo

## 26. Review XSS phải dùng T3 chưa có Review

1. D AVAILABLE, tạo Booking mới → D Accept → C lấy tid3. 2. D làm ARRIVED → IN_PROGRESS → PATCH location → COMPLETED như trang 8. 3. Không review ok trước; dùng ngay request XSS sau.

**POST** | Authorization: **Customer** | Body: raw / JSON

```
http://localhost:8080/api/trips/{tid3}/reviews
{"star":5,"c":"<script>alert('hack')</script>"}
```

**Đạt:** 201; Comment không chứa script thực thi (hiện tại thường rỗng sau sanitize). Không có popup/script chạy.

Nếu nhận 409 vì đã review: đó là chống trùng, **chưa chứng minh XSS**. Dùng Trip hoàn thành khác chưa review. Không sửa/xóa Review cũ để thử lại.

Bằng chứng mạnh: đối chiếu response sạch với dữ liệu đã lưu; không cần tạo frontend HTML hoặc thêm endpoint đọc Review.

## Khi gặp lỗi: sửa đúng nguyên nhân, không tạo dữ liệu hàng loạt

| Hiện tượng | Cách xử lý ngắn |
|---|---|
| 401 | Login lại; Bearer Token dán JWT thật, không kèm dấu nháy/chữ Bearer trong ô Token. Token sai/hết hạn không dùng tiếp. |
| 403 | Đúng role? C chỉ xem tài nguyên mình; A mới xem Driver admin; D phải sở hữu Offer/Trip. |
| 409 đăng ký | Email/phone/plate đã có. Dùng bộ mới; không xóa hồ sơ cũ. Nếu RPC gián đoạn, retry đúng body trước. |
| NO_DRIVER_FOUND | Kiểm tra BIKE/CAR, APPROVED, AVAILABLE, location và radius. Không tự chuyển tất cả CAR thành BIKE. |
| Trip chưa thấy | RabbitMQ bất đồng bộ: chờ ngắn, GET lại by-booking. Kiểm tra consumers nếu lâu; không tạo Booking thay thế. |
| Payment chưa eligible | Hoàn thành đúng Trip; kiểm tra cùng bid/pid và consumer Payment. Không callback để “ép” state. |
| Pay replay PENDING | Đúng snapshot gốc. GET Payment/Trip để xem COMPLETED/paid=true hiện tại. |
| 429 | Chờ 10-11 giây. Không đổi role/IP để né rate limit khi kiểm thử. |

# Kết quả chạy lại và ma trận 30 tiêu chí

Run **fb55e824ae96** tại HEAD **ac737ec**: **31/31 checks; 30/30 rubric**. Test dùng account/Driver/Booking riêng, rate limit thật; không reset DB/volume. Driver test được đưa OFFLINE sau lượt chạy. Các mục cấu trúc/Git/Docker được script và lần rà soát cấu hình kiểm tra ngoài HTTP.

| # | Tiêu chí | Bằng chứng | KQ / trang |
|---|---|---|---|
| 1 | Kiến trúc source | Source | PASS / 3 |
| 2 | Gitignore / ENV | Git/config | PASS / 3 |
| 3 | Nhiệm vụ Gateway | Source + HTTP | PASS / 3 |
| 4 | IPC gRPC + events | gRPC/event/DB | PASS / 3 |
| 5 | Compose 13 thành phần | Docker | PASS / 3 |
| 6 | Health / ready / 8 services | HTTP | PASS / 4 |
| 7 | RabbitMQ topology/consume | Broker + event | PASS / 4 |
| 8 | Gateway-only ingress | Compose/HTTP | PASS / 4 |
| 9 | Register Customer | HTTP/DB | PASS / 5 |
| 10 | Login Customer | HTTP | PASS / 5 |
| 11 | Get Customer | HTTP | PASS / 5 |
| 12 | Get Driver bằng Admin | HTTP | PASS / 5 |
| 13 | Nearby 1 km + paging | HTTP/PostGIS | PASS / 6 |
| 14 | Customer Booking history | HTTP/DB | PASS / 6 |
| 15 | Booking + một Offer | HTTP/event/DB | PASS / 6 |
| 16 | Accept + Trip + Driver info | HTTP/event/DB | PASS / 7 |
| 17 | Trip state + location | HTTP/event/DB | PASS / 8 |
| 18 | Cancel + 2 Notification | HTTP/Mongo | PASS / 10 |
| 19 | Payment mock + paid=true | HTTP/event/DB | PASS / 9 |
| 20 | Review + chống trùng | HTTP/DB | PASS / 10 |
| 21 | Driver phone / OTP | HTTP/DB | PASS / 11 |
| 22 | Admin duyệt hồ sơ | HTTP/event | PASS / 11 |
| 23 | Online / Offline | HTTP/DB | PASS / 12 |
| 24 | Hash / ciphertext tại DB | DB + source | PASS / 12 |
| 25 | SQL Injection | HTTP/SQL | PASS / 13 |
| 26 | XSS an toàn | HTTP/DB | PASS / 14 |
| 27 | JWT tampering | HTTP | PASS / 13 |
| 28 | Unauthorized 403 | HTTP | PASS / 13 |
| 29 | Redis 429 | HTTP/Redis | PASS / 13 |
| 30 | Payment replay | HTTP/event/DB | PASS / 9 |

ID đã nghiệm thu (chỉ tham khảo, không dùng lại làm chuyến mới): bid=55, tid=2744224, pid=49; cancel tid=2744226; XSS tid=2744227. Không có token/key mã hóa/mật khẩu hạ tầng trong PDF.

KHÔNG COLLECTION / KHÔNG ENVIRONMENT / KHÔNG SCRIPT

# Tờ nhắc nhanh khi gõ tay

Base: **http://localhost:8080/api**. Thay ID bằng số thật. C/A/D là ký hiệu role, không phải chuỗi Token. Trong Postman chọn raw/JSON cho body có JSON; còn lại none.

| Method + path sau /api | JWT | Body / ghi nhớ |
|---|---|---|
| POST /login | - | email,pw |
| POST /register | - | email,pw,name; phone tùy chọn |
| GET /customers/{cid} | C | none |
| GET /drivers/{did} | A | C cùng URL →403 |
| GET /drivers/nearby?lat=10.76&lng;=106.68 | A | r1/page1/limit5 mặc định |
| GET /customers/me/bookings?page=1&limit;=2 | C | none |
| POST /bookings | C | p,d; bỏ vt →BIKE |
| GET /notifications?limit=20 | C/D | none |
| GET /drivers/me/offers | D | none |
| POST /offers/{oid}/accept | D | none |
| GET /trips/by-booking/{bid} | C | none |
| PATCH /trips/{tid}/status | D | s: ARRIVED/IN_PROGRESS/COMPLETED |
| PATCH /trips/{tid}/location | D | lat,lng |
| POST /trips/{tid}/cancel | C | r:x (chuyến riêng) |
| GET /payments/by-booking/{bid} | C | none |
| POST /payments/{pid}/pay | C | Header Idempotency-Key; body none |
| POST /payments/{pid}/callback | C | s:COMPLETED |
| GET /payments/{pid} / trips/{tid} | C | đọc COMPLETED / paid=true |
| POST /trips/{tid}/reviews | C | star:5,c:ok |
| POST /drivers/otp | - | phone |
| POST /drivers/register | - | phone,email,pw,otp,name,plate |
| GET /drivers/pending-approval | A | page/limit khi cần |
| PATCH /drivers/{did}/approval | A | ap:APPROVED hoặc REJECTED |
| PUT /drivers/me/location | DN | lat,lng |
| PATCH /drivers/me/availability | DN | on:true / false |
| GET /vehicle-types | C | bấm 4 lần để thấy 429 |

**Nhớ 5 điều:** BIKE khác CAR; location sau IN_PROGRESS; hủy chuyến riêng; Pay trước callback; replay trả response Pay gốc. Không dùng lại P1 cho một Payment khác.

**Nguồn đối chiếu:** PHIEU_CHAM_PROJECT (1).pdf (6 trang), CAB-System-30-Tieu-Chi-Chi-Tiet-Final.pdf (16 trang); source/API/SRS hiện tại và script smoke của repository. Hướng dẫn đã viết lại theo API chạy thật; tài liệu tham khảo không phải lệnh cho phép thay đổi hệ thống.